ProveIt research engineering review

Shorter, clearer Lean proofs

DiophantineConsistency

Concrete refactorings, checked prototypes, and an adoption plan

Prepared for Vladimir Reshetnikov
26 October 2026

Source snapshot. `60e9c5d620c6c387d932876cfa2d9376f64ea56c`

Pinned environment. Lean 4.32.0; Mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`.

Main recommendation. Concentrate on repeated semantic interfaces: variable-layout transport, checked list folds, residual aggregation, finite certificate assembly, and source-register decoding. The repository already contains many of the generic mathematical theorems needed for this work. Making their clients use them consistently is the largest opportunity.

The accompanying archive contains a precise 40-line proof-only patch, standalone Lean prototypes, source measurements, and reproducible checks. Component reductions are distinguished from unmeasured whole-project savings.

Scope. This is a proposal and prototype review of existing proofs. It does not claim a new consistency theorem, a completed integration of the current polynomial with its machine interpretation, or a measured global compilation speedup. Existing repository sources were kept unchanged.

Contents

1 Findings and priorities 2

2 Reviewed snapshot and measurements 3

3 Immediate proof-only improvement: use Lean's list API 3

4 Witness layouts: prove the transport once 4

5 Checked histories and numeric scanners 5

6 Residual aggregation and generic gate semantics 7

7 Generated circuit proofs: use the existing certificates 9

8 Native arithmetic: exact identities and shared geometry 10

9 Carry certificates and finite data assembly 11

10 Object-language proofs: centralize the semantic boundary 12

11 Adoption plan and criteria 13

12 Validation, limitations, and reproducibility 14

Primary sources 16

1 Findings and priorities

The best route to shorter proofs is to make the repeated mathematical interfaces explicit, then retain the original executable objects and prove their correctness through those interfaces. In this project, proof length, the size of a represented program, the size of a polynomial, and the cost of kernel checking are separate quantities. A successful proof refactor can leave all program and polynomial data definitionally unchanged.

There are three especially promising levels of work. First, small proof-only changes can immediately remove local induction and transport boilerplate. Second, a few reusable semantic APIs can replace the same list, witness, and sum-of-squares arguments in several modules. Third, the generated circuit proof files can use the certificate machinery already proved in the repository, substantially reducing their materialized proof source while retaining named semantic intermediate results.

Order	Proposal	Evidence or deliverable	Effort
1	Replace seven list-lookup inductions	Exact patch; 40 physical lines saved	Small
2	Share witness-layout semantics	Compact constructors with expression-equality checks	Small
3	Share checked-history folds and scan recurrences	Six generic laws, checked in Lean	Small–medium
4	Centralize executable sum-of-squares semantics	Three repeated evaluation proofs; shared degree/cost API	Small–medium
5	Use parent power certificates in generated circuits	193 power declarations; 1,538-line old component	Medium
6	Specialize local DAG equations on demand	999 existing per-index wrappers	Medium
7	Replace half-radix case splits by an exact identity	Stronger generic arithmetic lemmas, checked	Small
8	Abstract finite chunk assembly	Repeated decoder and rail dispatch; retain checked leaves	Medium
9	Share physical-field residues and source facts	Literal duplicate residues and repeated Pell row decoding	Medium
10	Derive carry recurrence from a smaller certificate	Existing digit bound plus exact decomposition suffice	Medium
11	Factor object-language proof transport	27 completeness calls; reusable sealed-validity bridge	Medium
12	Move projection laws and generic gate proofs earlier	Existing APIs blocked by dependency direction	Medium

The order reflects risk and ease of review, rather than a claim that the earliest item yields the largest line reduction. The power certificate work offers the largest concrete component compression; the witness and scanner interfaces offer the strongest improvement to day-to-day proof maintenance. Several further ideas are developed below: a dynamic-seed primitive-recursive iterator, Boolean reflection, a small copied-field interface, and disciplined proof profiling.

2 Reviewed snapshot and measurements

The review uses the repository's `main` commit `60e9c5d620c6c387d932876cfa2d9376f64ea56c`. The checkout was pinned before source inspection. The active library rooted at `Computability/DiophantineConsistency/Lean` contains 546 Lean files and 73,971 physical lines. Of these, a filename/path classification identifies 220 check or audit files and 326 other files, including the aggregate import module. The latter group contains 66,857 physical lines. The whole requested directory contains 711 tracked Lean files; 165 lie in historical candidate or provenance material outside the active library. They are not included in the active-library line totals or treated as independent refactoring targets.

Active-library measurement	Value
Lean files	546
Physical source lines	73,971
Nonempty code lines after masking comments/strings	59,832
Occurrences of `theorem` in code	4,970
Occurrences of `omega` / `nlinarith`	1,681 / 563
Occurrences of `ring` / `decide`	686 / 4,080
`maxHeartbeats` / `maxRecDepth` settings	134 / 177

These are lexical measurements, not kernel proof-term sizes or timings. The comment-aware scan found no `sorry`, `admit`, `native_decide`, or explicit axiom declarations in the active source. That observation does not replace transitive axiom audits or compilation. The supplied `scripts/inventory.py`, CSV files, and JSON inventory make the counting method and file hashes inspectable.

The largest proof-source targets are:

Module	Physical lines	Code lines
`SplitCircuitSteps59.lean`	3,445	2,604
`GapCircuitSteps66.lean`	3,122	2,375
`SplitCircuit59.lean`	1,286	1,223
`SplitJointCircuit59.lean`	1,252	1,189
`GapCircuit66.lean`	1,226	1,163
`ShortSystem.lean`	1,117	1,036

The raw circuit tables and the correctness proofs serving them have different roles. Literal data should remain reviewable. The primary target is the repetitive proof layer around that data.

3 Immediate proof-only improvement: use Lean's list API

Seven local proofs repeat induction on a list to connect optional indexing with membership. Lean 4.32.0 already supplies both directions, with the required types:

```
theorem lookup_mem {α : Type} {xs : List α} {i : Nat} {a : α}
    (h : xs[i]? = some a) : a ∈ xs :=
  List.mem_of_getElem? h

theorem exists_lookup {α : Type} (xs : List α) (a : α)
    (h : a ∈ xs) : ∃ i : Nat, xs[i]? = some a :=
  List.getElem?_of_mem h
```

Module	Proofs	Lines removed
`AnnotatedRename`	1	5
`AnnotatedBound`	1	5
`AnnotatedProofChecker`	1	8
`AnnotatedAssumption`	2	11
`AnnotatedRuleSide`	2	11
Total	7	40

The exact sites are `AnnotatedRename.lean` (lines 110–117), `AnnotatedBound.lean` (lines 50–57), `AnnotatedProofChecker.lean` (lines 177–187), `AnnotatedAssumption.lean` (lines 40–57), and `AnnotatedRuleSide.lean` (lines 18–35). The supplied `patches/lookup-api.patch` replaces proof bodies only. All declaration names, statements, and numeric definitions remain identical. Its two polymorphic replacement proofs compile under the pinned compiler, and `gitapply--check` succeeds against the snapshot. The complete patched project modules were not replayed in this review; that distinction is recorded in the validation evidence.

This is an excellent first change because its scope is small and its benefit is exactly measurable. It also illustrates a broader search method: inspect locally reproved structural lemmas before designing new automation. Existing library facts often remove an entire proof instead of merely shortening its tactic script.

4 Witness layouts: prove the transport once

4.1 The repeated argument

`Representation.lean` (lines 155–256) repeatedly proves that renaming input and witness coordinates commutes with evaluation. In particular, `Rep.conjunction` and `Rep.disjunction` each introduce the same left and right evaluation equalities, using function extensionality and case analysis. `reindex` and `existsBlock` contain closely related arguments. A private one-sided helper also exists in `QuarticCompiler.lean` (lines 95–98); the proposal is to make the general semantic law available at the representation layer.

The central law is:

```
natAssignment v w ∘ Sum.map f g =
  natAssignment (v ∘ f) (w ∘ g)
```

Its proof is just `funexti;casesi<;>rfl`. Together with the existing `Expr.eval_rename`, it eliminates the repeated left/right transport proofs.

The second law is the existential equivalence between a single concatenated witness block and its two components:

```
(∃ w : Fin (m + n) → Nat,
  P (w ∘ Fin.castAdd n) (w ∘ Fin.natAdd m)) ↔
    ∃ a b, P a b
```

The forward map projects the two blocks. The reverse map uses `Fin.addCases`. This works when either block has length zero, so special empty-witness cases are unnecessary.

4.2 Preserve the executable data

The supplied `WitnessLayout.lean` defines compact conjunction and disjunction constructors. Their witness counts are unchanged and their expression fields are definitionally equal to the originals; the file contains explicit `rfl` checks of expression equality. Thus the proposal simplifies correctness proofs without changing the represented polynomials, variable order, or number of witnesses.

The original two constructor declarations occupy 59 physical lines; the compact versions occupy 32. This comparison excludes the shared helper API. Its one-time cost must be included in a net repository comparison, and it should be amortized across `reindex`, `existsBlock`, and further layout clients. The immediate gain is conceptual: arithmetic correctness is no longer interrupted by repeated function-coordinate bookkeeping.

4.3 Move existing projection lemmas to their natural home

The desired witness-count simplification lemmas already exist in `RepresentationSize.lean` (lines 136–147), with graph analogues at lines 193–209. However, `DirectComposition.lean` (lines 59–68) still unfolds whole constructors to compute counts. It cannot import the size module without reversing the present dependency direction.

Move the small `rfl` projection lemmas next to the constructors or into a lightweight module that imports only the constructors. The size module can then concentrate on global bounds. This changes which facts are available early, rather than adding duplicate facts. It also prevents the simplifier from examining large expression and proof fields merely to calculate a natural-number projection.

5 Checked histories and numeric scanners

5.1 One invariant-preserving history fold

The list processors in `AnnotatedRename`, `AnnotatedBound`, and `AnnotatedProofChecker` have the same recursive shape: validate the next line against a history; prepend the emitted result; continue; or reject. Rename and bound rows emit themselves; proof lines emit a judgment. The appropriate abstraction therefore has two types and two functions:

```
def run (valid : Line → List Judgment → Bool)
    (emit : Line → Judgment) :
    List Line → List Judgment → Option (List Judgment)
  | [], seen => some seen
  | line :: lines, seen =>
      if valid line seen then
        run valid emit lines (emit line :: seen)
      else none
```

The generic soundness proof assumes exactly the local invariant step: if all existing judgments satisfy P, and a line passes validation, then its emitted judgment satisfies P. Induction on the remaining lines proves the invariant for the final history.

The duplicated clusters are `AnnotatedRename.lean` (lines 196–235), `AnnotatedBound.lean` (lines 136–175), and `AnnotatedProofChecker.lean` (lines 206–250). They contain `run_append`, `run_sound`, `Produces.append`, and `Produces.step`. The supplied `ValidatedFold.lean` proves the shared versions. The existing `Produces` interface is useful; its implementation and laws should become shared while public module-local names remain thin specializations. Formula-specific soundness remains in the current row or line soundness theorem.

A conservative migration keeps the old executable `run` definitions and proves their equality with the common fold. A later migration may replace those definitions by transparent specializations, but only after checking any syntax-sensitive size claims. Extensional equality of programs does not imply equality of their encoded abstract syntax trees.

5.2 Separate recurrence from encoding

Five scanner modules independently establish the same iterator equations: `AnnotatedRenameScan`, `AnnotatedBoundScan`, `AnnotatedLineScan`, `AnnotatedAxiomContext`, and `AnnotatedContext`. Their `scan_succ` and fixed-terminal-state proofs depend on iteration, not on pairing codes or formulas.

For an arbitrary state type, define

```
def scan (step : State → State) (fuel : Nat)
    (initial : State) : State :=
  fuel.rec initial (fun _ acc => step acc)
```

Then prove once:

```
scan step (fuel + 1) initial = scan step fuel (step initial)
step fixed = fixed → scan step fuel fixed = fixed
```

The two supplied recurrence proofs compile without any axioms. They can replace the repeated induction proofs while preserving the old numeric definitions through transparent unfolding.

The next stage is to share the simulation between encoded lists and typed folds. Concrete targets are `AnnotatedRenameScan.lean` (lines 71–87), `AnnotatedBoundScan.lean` (lines 67–83), `AnnotatedLineScan.lean` (lines 132–152), and `AnnotatedAxiomContext.lean` (lines 130–151). The common simulation should take a `step_cons` equation, an acceptance test, a history update, and terminal/rejection encodings.

Two details are essential. Ordinary histories use zero as the empty code, whereas an empty quoted context uses one. Also, correctness for raw codes must continue to cover malformed inputs and arbitrary supplied fuel. Proving only the behavior of well-typed encodings would weaken that contract.

5.3 A better primitive-recursive iterator interface

`DirectScalarArithmetic.lean` (lines 103–107) already provides `iterate`, with a fixed numeric seed. The useful extension is `iterateFrom`, accepting realized seed and fuel functions, plus a parameterized step that can still access the original input. The repeated constructions occur in `AnnotatedContextPR.lean` (lines 47–54), `AnnotatedBoundScan.lean` (lines 144–158), `AnnotatedLineScan.lean` (lines 231–245), and `AnnotatedRenameScanPR.lean` (lines 48–75).

The result should remain an explicit `Rf` object with supplied program data. A bare existence theorem `Nat.Primrecf` would lose the data needed by the compiler and its size accounting. The expensive validator is deliberately used once in `AnnotatedLineScan.lean` (lines 201–226); preserve that sharing. Require equality of the constructed `.val` program, or recheck the exact node-count theorems, before claiming unchanged encoded size.

5.4 Share tag dispatch and Boolean reflection

`AnnotatedRenameLocal.lean` (lines 63–87) and `AnnotatedBoundLocal.lean` (lines 55–79) repeat the same tag partition: 0–1, 2, 3–5, 6–7, and rejection for tags at least 8. A single arithmetic dispatch lemma can certify that the threshold implementation matches this partition, parameterized by the four branch results.

The scanner interface can also be strengthened from an acceptance equivalence to an exact value equation. Combine `check=1↔valid=true` and `check=0vcheck=1` into `check=truthvalid`. This removes repeated true/false reasoning in the two `step_cons` proofs. The range premise is essential: the acceptance equivalence alone allows a rejecting numeric result of two.

6 Residual aggregation and generic gate semantics

6.1 One executable sum-of-squares API

Three modules contain equivalent private `eval_sumExpressions` proofs: `DAGQuartic.lean` (lines 83–86), `NaturalDAGQuartic.lean` (lines 110–113), and `QuadraticRegisterSystem.lean` (lines 70–73). Each then proves that a finite sum of squared residuals is zero exactly when each residual is zero, using the same `List.ofFn`, finite-sum conversion, and ordered-ring lemma. Their degree proofs also repeat the same sum and product bounds.

Introduce a public expression constructor and three reusable theorems:

```
def sumSquares (rows : Fin n → Expr σ) : Expr σ :=
  QuarticSyntax.sumExpressions
    (List.ofFn fun i => rows i * rows i)

(sumSquares rows).eval v = 0 ↔
  ∀ i, (rows i).eval v = 0

(∀ i, (rows i).toPolynomial.totalDegree ≤ d) →
  (sumSquares rows).toPolynomial.totalDegree ≤ 2 * d

(∀ e ∈ es, e.binaryOps ≤ b) →
  (QuarticSyntax.sumExpressions es).binaryOps ≤
    (b + 1) * es.length
```

The first equivalence is over integers, or another suitable ordered structure where a sum of squares vanishes termwise. It is not valid over an arbitrary commutative ring. The supplied prototype deliberately uses integer evaluation. The degree statement is an upper bound, so it does not claim an exact degree in the presence of cancellation.

The cost generalization also absorbs the existing special case with row cost five in `QuarticCost.lean` (lines 60–65), and the general argument repeated in `DAGQuartic`. Because the expression uses the same ordered list of squared residuals as before, clients can retain the exact syntax rather than replacing it by an algebraically equivalent polynomial.

Signed witness reconstruction and natural witness reconstruction need different hypotheses. Share residual aggregation first; a fully generic signed/natural encoding framework should be attempted only if its instantiation overhead is smaller than the duplication it replaces.

6.2 The semiring theorem already subsumes the natural induction

`QuarticCompiler.lean` (lines 101–167) proves natural gate completeness and soundness by syntax induction. `GatesSemiring.lean` (lines 72–145) proves the corresponding results over an arbitrary commutative semiring, covering the natural case. The two expression interpretations are already connected by `Expr.eval_eq_zero_iff_splitNatural` and `NatExpr.evalSemiring_nat`.

The useful refactor is architectural. Extract gate data, wire indexing, roots, residual syntax, and value definitions into a lightweight module. Prove generic gate semantics above it. Then derive the natural-number results through the interpretation bridge, retaining their public names. Currently `GatesSemiring` imports `QuarticCompiler`; simply calling the generic theorem from the compiler would introduce a cycle. Moving the data is the prerequisite.

Likewise, `QuadraticRegisterSystem.lean` (lines 18–33) implements an unweighted degree and its polynomial bound, while `WeightedExpressionDegree.lean` (lines 37–62) already proves the generic weighted result. Constant weight one supplies the old bound. Keep an abbreviation or compatibility equality if downstream proofs depend on the old recursive equations. The polynomial-renaming theorem in `QuarticCompiler.Gates` belongs in the expression API for the same reason: basic syntax laws should be available without importing a compiler.

7 Generated circuit proofs: use the existing certificates

7.1 Where the source volume comes from

`GapCircuitSteps66.lean` contains 484 local-step theorems and 86 power theorems. `SplitCircuitSteps59.lean` contains 515 step theorems and 107 power theorems. Together they occupy 6,567 physical lines. Every local-step theorem already calls the generic `ArithmeticDAG.run_node_equation`; that theorem is not missing. The repository also already has `run_eq_of_node_equations`, `value_eq_of_node_equations`, `run_power`, and a certified power-sharing rewrite. The opportunity is to adopt these existing interfaces consistently at the generated clients.

7.2 Replace 193 power chains by parent exponent certificates

The old power sections occupy 685 and 853 lines respectively, for a combined 1,538 lines. Each chain follows multiplication nodes and proves an exponent identity. The existing `run_power` theorem in `ArithmeticDAGPowers.lean` (lines 26–46) abstracts exactly this argument.

Assign a natural exponent label $\ell(i)$ to each node. Label zero means that the node is outside the certified pure-power fragment. The distinguished base has label one. Each labeled non-base node must be a multiplication of earlier labeled nodes, with

$$\ell(i) = \ell(a) + \ell(b).$$

The literal node table is finite, so these local conditions are checked by `decide+kernel`. Topology and the existing theorem then yield

$$\text{value}(i) = \text{value}(\text{base})^{\ell(i)}.$$

The supplied `GapPowerCertificate.lean` and `SplitPowerCertificate.lean` keep the original `naturalNodes` exactly. They are proof certificates for those circuits, not optimized replacement circuits. Their initial source representation occupies 117 lines including label tables and examples, compared with the 1,538-line old power component. This is a component comparison; it is not a measured integrated project reduction. Existing named callers must be migrated or retained as thin compatibility wrappers. The supplied metrics record the concrete versions included in the archive.

For Split, all 515 entries of the proposed label table agree with the old-label table already present in `SplitJointPower59.lean` (lines 64–78). Move that parent certificate to a small shared module rather than importing an unrelated child circuit solely to reuse its labels. The Python generator checks labels against the actual multiplication nodes as a diagnostic. The Lean certificate is the proof.

Do not infer a compilation speedup from the shorter source. One finite certificate can still be expensive to reduce. Compare actual replay time, proof object size, and peak memory before choosing its final chunking.

7.3 Specialize low-level node equations at use sites

The supplied `CircuitDispatch.lean` wraps the generic node equation in one `steppxihi` theorem. At a concrete index, its type reduces to the same scalar equation as `natural_step_i`. A short example uses six selected node equations to prove the base identity $p_0(x_0 + 1)^2$.

This creates an alternative to eagerly declaring all 999 low-level wrappers. However, replacing every short theorem name by a verbose specialization can make clients harder to read. The practical com-

promise is a small local elaboration helper or generator that emits applications of the existing theorem, together with named results at meaningful register boundaries.

Do not make unrestricted node expansion a global simplification rule. Unfolding all shared registers recursively can turn a DAG into a large expression tree. The named base, kernel, mask, rank, and source bindings in `GapCircuitRegisters66` and `GapCircuitRecipe66` are useful proof boundaries. Keep those boundaries, and reduce the repetitive low-level transport beneath them.

8 Native arithmetic: exact identities and shared geometry

8.1 A stronger half-radix lemma

For all natural a, b and positive s,

$$\boxed{(sa) \bmod (2s) + \left\lfloor \frac{sb}{2s} \right\rfloor = s(a \bmod 2) + \left\lfloor \frac{b}{2} \right\rfloor .}$$

The proof cancels a common positive factor in the modulus and quotient. The Lean proof uses `Nat.mul_mod_mul_left` and `Nat.mul_div_mul_left`, after commuting $2s$.

Since $a \bmod 2 \le 1$, the expression is at most $s + 1$ whenever $b \le 3$, with no bound on a. If s is even, the expression's parity equals $\lfloor b/2 \rfloor$ under the same bound on b. Without that bound, the parity is instead $\lfloor b/2 \rfloor \bmod 2$.

This replaces the four-case bound proof in `NativeCyclicRotation.lean` (lines 100–116) and the sixteen combinations used for parity at lines 160–165. The original proof bodies occupy 15 and four nonblank lines; proposed wrappers are one-line applications of the generic lemmas. Their helper cost must still be counted, so this is a local simplification and strengthening, not automatically a net line saving across the whole project.

`NativeRefactorCore.lean` checks the identity, bound, and parity lemmas using only Lean's core arithmetic infrastructure. The supplied `NativeRotationWrappers.lean` shows their intended applications to the actual `shiftDigits`; its integration status is stated in the validation ledger. The already generic Boolean rotation API in `NativeBooleanRotation` remains useful and does not need replacement.

8.2 Share physical-field residue calculations

The addition/subtraction residue proofs in `CurrentClause2011.lean` (lines 27–39) are repeated privately in `CurrentReflectedGeometry2011.lean` (lines 74–86). A small `CyclicFieldArithmetic` module should own these facts for total length NL and field width L. For an aligned base b, field offset $e < L$, and arbitrary natural shift t, a useful normal form is

$$(b + e - t).\mathrm{val} \bmod L = (L + e - (t \bmod L)) \bmod L.$$

The positional subtraction on the left is in $\mathbb{Z}/(NL)\mathbb{Z}$. The bounded natural subtraction on the right is safe because $t \bmod L < L$. This is already the substance of `CurrentCellTranslation2011.lean` (lines 84–94); extract it downward instead of importing that later module backward and creating a cycle.

The clause and marker convolution proofs both calculate the residues of shifts 18 and 2020, then use support exclusion. A shared residue normal form and support-zero helper would replace repeated arithmetic with one instantiation and a small numerical check. The origin-exclusion step in the corresponding mask lemmas can also be shared.

A particularly direct reuse is available for reflected coefficients. `CurrentClause2011.lean` (lines 122–130) proves the residue modulo 2048. `CurrentEarlyPadding2011.lean` (lines 95–107) reproves the corresponding modulo-16 result by cases. After sharing the identical selector-unit definition, project the stronger congruence using `.of_dvd(bydecide:16|2048)`.

Preserve arbitrary shifts, wraparound, and $N = 1$. A later equivalence between aligned cells and `ZModN` could simplify reindexing, but the immediate extraction does not require a new coordinate representation.

8.3 Decode source-register equations once

Several Pell arguments reconstruct the same raw arithmetic equations:

$$q = 1 + J\beta, \qquad L = q^2 - 1, \qquad N = q(1 + L) = q^3, \qquad X = qw.$$

Concrete repetitions occur in `source_Y_ge_two`, `source_rank_eq_target`, and `source_scale_eq` in `NativePellBootstrap.lean` (lines 95–115, 415–436, 498–518), and in `NativePellPower.lean` (lines 127–143). Main and auxiliary classifications also decode related norm rows separately.

Extend the existing source-register helper approach to a small set of named arithmetic facts proved once from the literal rows. State the primitive identities over integers and derive natural-number corollaries with explicit nonnegativity premises where subtraction is involved. Pell arguments then consume semantic equations, with fewer row unfoldings and casts.

Keep raw row equations, positivity, port bounds, Pell classification, and rank recovery in separate interfaces. Bundling all of them together can silently add assumptions to clients. The existing `PortBox` already packages port inequalities; a differently named replacement adds no value.

9 Carry certificates and finite data assembly

9.1 Derive recurrence instead of storing it twice

The existing `CurrentPolynomialWindow2011.Facts` record stores a digit bound, a quotient recurrence, and an exact local decomposition. The quotient recurrence follows from the other two. In natural arithmetic,

$$u = d + rk, \quad d < r \quad \Longrightarrow \quad k = \lfloor u/r \rfloor, \quad d = u \bmod r.$$

The strict digit bound already implies $r > 0$. No extra positivity assumption is needed in the generic implication.

The checked `CarryCertificate` prototype stores carry bounds, digit bounds, and decomposition, indexed by the existing arrays and predecessor function. Its `recurrence` theorem is a derived method. In the current window, take $u_i = \text{convolution}_i + \text{rotation}_i + k_{i-1}$.

This reduces repeated argument lists in `CurrentPhaseParity2011.lean` (lines 92–163) and the parallel fields in `CurrentPolynomialWindow2011.lean` (lines 27–65). It also makes explicit which facts are independent. Construct the record using the existing normalization theorem obtained from the actual polynomial. A convenient record must not become a new unproved assumption in place of that construction.

Preserve minimal low-level statements. Some carry-reset lemmas need only decomposition and carry bounds, without a digit bound or recurrence. Keep those lemmas general and offer record-based

wrappers to larger clients. Changing every theorem to demand a large certificate would shorten notation while strengthening hypotheses unnecessarily.

9.2 Keep checked finite chunks; share their assembly

`WangTiles942Data.lean` (lines 210–321) proves `action_eq_decoder` by repeatedly splitting an index into successive 16-entry ranges. Its rail-cache counterpart repeats the pattern at lines 439–512. Each branch reconstructs a local index, proves bounds, and transports an already checked chunk equality.

The generic dispatcher in `NativeRefactorCore.lean` uses Euclidean division below Bq, and subtraction in the final tail:

$$i = B\lfloor i/B \rfloor + (i \bmod B), \qquad i < Bq + r.$$

It combines q complete blocks of size B and a final block of size r. The action table uses $(B, q, r) = (16, 26, 14)$; the rail table uses $(16, 13, 8)$. A small finite case selection can then choose the already authenticated chunk theorem. An estimated 90–120 lines could be removed across these two dispatchers, after helper overhead; this is an estimate awaiting an integrated patch.

The same principle applies to the hand-built sum assembly in `CurrentKernel2011.lean` (lines 156–419). Keep the manageable independently checked chunks, but describe their combination as data and apply one proved aggregation theorem. A list of partial totals is enough to generate a balanced assembly tree if reduction cost warrants it.

Do not replace kernel-checked leaves with `native_decide`. Native evaluation adds a compiler-dependent axiom to the proof's trust boundary, as documented by the official Lean reference. Nor should one assume that a single enormous `decide+kernel` call is cheaper than the present chunks. Compression should remove repeated assembly logic while retaining the ability to control kernel reduction size.

9.3 One copied-field bound interface

`CurrentCellLabels2011.lean` (lines 66–77, 283–299) reconstructs the same four copied-field bounds, using offsets and widths $(978, 8), (960, 9), (969, 8), (942, 9)$. The convolution-word proof repeats the corresponding four block instantiations.

A small first step is `copiedFields_bounded`, returning the four facts once. If more clients emerge, use a finite field descriptor with offset, width, and packed shift, plus a generic field evaluation/bound theorem. Preserve the actual order: packed fields are west, north, east, south, whereas the `ColorsBounded` conclusion orders west, east, north, south. A broad tuple refactor must handle that difference explicitly.

10 Object-language proofs: centralize the semantic boundary

10.1 A shared bridge from arbitrary models to sealed derivations

There are 27 occurrences of `completeness_inf_context` in 18 `Object*.lean` files. Twenty-five follow the direct pattern of extracting a model's ZF axioms, proving semantic validity, and applying universal closure. Examples include `ObjectNaturalExpression.lean` (lines 181–202), `ObjectMultiplication.lean` (lines 198–215), and `ObjectAdditionLaws.lean` (lines 183–206).

The following is the intended interface, not an assertion that this particular integration theorem has been replayed:

```
theorem zfc_proves_seal_of_zf_valid (φ : Form)
    (hφ : ∀ (V : Type) (mem : V → V → Prop),
      ZFAxioms mem → ∀ ee : Nat → V, Sat mem ee φ) :
    BProv ZFCax_s [] (sealF φ)
```

Its proof owns completeness, the extraction of axioms from a ZFC model, and `seal_valid`. Each arithmetic client supplies only its semantic validity theorem. The private model-extraction proof in `ObjectArithmetic.lean` (lines 143–164) and public copy in `ObjectMultiplication.lean` (lines 169–183) can be consolidated at a low dependency layer. A later upstream helper already exists, but importing the much heavier bounded-consistency layer into early arithmetic merely to save a few lines would be a poor dependency tradeoff.

Keep the quantification over arbitrary models visible. Their internal natural numbers may be non-standard. Replacing their arithmetic by Lean's ordinary `Nat` would change the theorem. The existing `ObjectOmegaSemiring`, `ObjectNaturalOrderedSemiring`, and `ObjectNaturalSemiring.evalRel_iff` already provide the relevant internal algebra; adding semiring instances is not a new recommendation.

Likewise, `ObjectNaturalExpression.evalRel_unique` currently has no natural-input guard. A typed semiring specialization with stronger input hypotheses does not subsume that theorem. Preserve its current generality separately.

10.2 Generalize closed-sentence transport across arity

`ObjectQuarticCompiler.lean` (lines 100–116) defines a local `closed` helper for the source arity and then repeats the proof for the target arity. Quantify over an arbitrary arity k, with argument `(f:Expr(Fink))`. The target becomes another application of the same theorem.

Preferably expose the result as `ObjectPolynomial.sat_noZeroForm_at_iff`, parallel to the existing environment-specific consistency lemmas in `ObjectConsistency` and `ObjectCodeSeal`. This is a narrow reusable interface for closed formulas; it does not require redesigning the object language.

11 Adoption plan and criteria

11.1 Stage 1: proof-only edits and shared laws

Start with the lookup patch. Next introduce witness-layout transport, sum-of-squares aggregation, and generic scan recurrences. Preserve old declaration names and executable definitions, adding compatibility lemmas where needed. Check the expression and program data before spending time on shorter tactic scripts: `rfl` equality is the strongest convenient result when it is available.

Each review should report old and new source lines including the shared helper, the affected statements, and the axiom set of the public endpoints. Do not count a common helper as free. Some APIs pay back only after their second or third client migrates.

11.2 Stage 2: reuse certificates and reduce generated wrappers

Move parent exponent certificates into reusable modules and replace power chains in the old generated proofs. Retain thin aliases during migration if that keeps downstream changes small. Then introduce

scoped node specialization beneath the named semantic register lemmas. Measure source bytes as well as lines, because dense certificate tables make physical line counts sensitive to formatting.

For finite tables, keep the checked leaf size initially and change only the dispatcher or sum assembly. Rechunking and changing the mathematical representation should be separate changes, so a performance regression has an identifiable cause.

11.3 Stage 3: reorganize dependencies and object transport

Move basic constructor projection laws down the import graph. Split gate data from generic semantics before replacing natural syntax inductions. Extract shared cyclic-field arithmetic, source-register equations, and the arbitrary-model bridge into appropriately early modules. Then migrate clients incrementally, preserving minimal assumptions at low levels.

Avoid importing the aggregate `DiophantineConsistency` module into a small proof that needs only one or two leaf APIs. This is an engineering recommendation based on the aggregate's broad import list, not a measured claim about any particular build's bottleneck.

11.4 Profile a concrete bottleneck rather than increasing budgets

The source contains many heartbeat and recursion-depth settings. Those are candidate locations to inspect, not proof that those files are slow. For a representative theorem, record elaboration time, kernel-checking cost where available, peak memory, and generated object size under the same compiler, cache state, and concurrency. Use the pinned compiler's profiling tools and narrow the measurement to a declaration or module.

Prefer stable semantic rewrites and a small `simponly` vocabulary at a large interface. During development, use search or suggested simplifications to discover the right facts, then keep the resulting proof explicit. Global simplification rules that unfold full circuits, entire source-row systems, or encoded syntax can erase the very sharing that makes the proof tractable.

12 Validation, limitations, and reproducibility

Six supplied Lean files passed with Lean 4.32.0. The successful audited declarations use only `propext`, `Classical.choice`, and `Quot.sound`; several use fewer, and the generic scanner recurrence lemmas use none. No successful audit contains `sorryAx` or a native-evaluation axiom.

File	What was checked
`LookupAPI.lean`	Two generic replacements for list-lookup induction, using the pinned core library.
`ValidatedFold.lean`	Checked-history append and soundness, fragment composition, and two generic iterator laws.
`NativeRefactorCore.lean`	Exact half-radix arithmetic, bounds/parity, finite chunk dispatch, quotient/remainder consequences, and carry recurrence.
`WitnessLayout.lean`	Compact constructors importing the original `Representation`; expression fields agree with the originals by `rfl`.

File	What was checked
`SplitPowerCertificate.lean`	Original compiled `SplitCircuit59`; 107 local power labels, the uniform power theorem, and exponent-1201 example.
`GapExactDataFixture.lean`	Exact original 484-node data, topology, 86 power labels, uniform power theorem, exponent-2200 example, and scoped node dispatch/base identity.

The original `GapCircuit66` module was terminated with exit code 137 during the focused dependency build. The original `SplitCircuit59` module and the generic DAG semantic/power modules completed successfully. For Gap, the fallback fixture copies the complete `naturalNodes` definition byte-for-byte: 8,078 definition bytes, with extraction offsets and SHA-256 hashes recorded in `evidence/gap_fixture_source.json`. Only the length, topology, power and dispatch obligations needed by the prototype are replayed there. This is a narrower validation than building the full original module.

The production-import Gap power and dispatch candidates therefore have exact-data fixture validation, not a successful replay through the full original Gap import. `CompilerCommon.lean`, `NativeRotationWrappers.lean`, and `SemiringSpecialization.lean` remain integration proposals without full module replay. They are kept in a separate `proposals/` folder. The exact lookup patch passes applicability checking; the five entire patched project modules have not been compiled.

The final checked witness code supplies the existential-layout predicate explicitly to rewriting. An initially implicit higher-order rewrite did not match. Similarly, the dispatch prototype needs scoped node-table, `Node.evalAt`, list-index and `Fin.mk_zero` reductions, while leaving the evaluator itself opaque. These are concrete elaboration details resolved in the successful prototypes, not reasons to introduce unrestricted global simplification.

All per-file logs and the machine-readable validation manifest are included. Reported wall times are environment-specific observations; they do not establish a performance improvement over the old proofs.

12.1 What the checks establish

An exit-zero Lean compilation checks the supplied declaration's proof in its stated environment. The expression-equality examples in the witness prototype additionally check the proposed constructors' literal expression data. The power prototypes, when marked passed in the ledger, use the original circuit definitions and validate their exponent-label conditions in Lean. Python's parallel label checks are diagnostics and do not enlarge the trusted proof base.

The lookup patch is distinct: the two generic proof terms compile and the patch applies cleanly, but the five entire modified modules have not been replayed. The other large refactorings are source-grounded designs or standalone theorem prototypes. No whole-project shortened variant has been built, and no whole-project speedup or reduction percentage is claimed.

12.2 Logical boundaries to retain

Several source facts constrain any simplification:

- `NativeFieldCycles2011.offset19_excludes_510` excludes value 510. The genuine 511 self-loop satisfies its assumptions and is tested in the existing checks. A replacement claiming that every nonzero cycle is impossible would be false.

- The offset-20 exclusion applies to an arbitrary function; it must not gain a finiteness or permutation hypothesis merely to share a wrapper.
- Cell-translation facts include wrapping, arbitrary large shifts, and the one-cell case. Keep these concrete regression checks.
- The maintained current polynomial's arithmetic window and the pending copy/label/machine composition are different proof obligations. Refactoring arithmetic interfaces does not complete the latter.
- Ordinary host-level arithmetic, arbitrary-model internal naturals, and syntactic ZFC derivability remain separate semantic layers.

These are specific properties of the reviewed source, not generic restrictions on proof automation. They identify where a superficially shorter proof could inadvertently weaken the intended result.

12.3 Archive contents and replay

The archive includes this `report.tex`, its compiled PDF, the Lean prototypes, the proof-only patch, source inventories and dependency edges, label-certificate generation and checking scripts, and validation logs. `README.txt` explains the exact commands and distinguishes verified modules from integration sketches.

To reproduce the source inventory, run the supplied Python script against a checkout at the pinned commit. To reproduce Lean checks, use the repository's `lean-toolchain` and `lake-manifest.json`; the replay script builds only the imports needed by the selected prototypes. The report can be rebuilt with XeLaTeX twice. All source links in the PDF are pinned to the reviewed commit, so later repository changes do not silently change the evidence.

Primary sources

The principal evidence is the source linked throughout this report at `60e9c5d620c6c387d932876cfa2d9376f64ea56c`. The repository's `lean-toolchain` and `lake-manifest.json` identify the replay environment. The project status record and the comments of the current endpoint modules delimit the still-open composition work.

For the trust-boundary distinction between kernel reduction and native evaluation, see the official Lean reference, Axioms, and Tactic Reference. Those online pages track the current manual; all executable evidence recorded as successful here was checked against Lean 4.32.0 as recorded in the ledger.